



SUN™ TECH DAYS

A DEVELOPER CONFERENCE 2001•2002

Security for Web Services

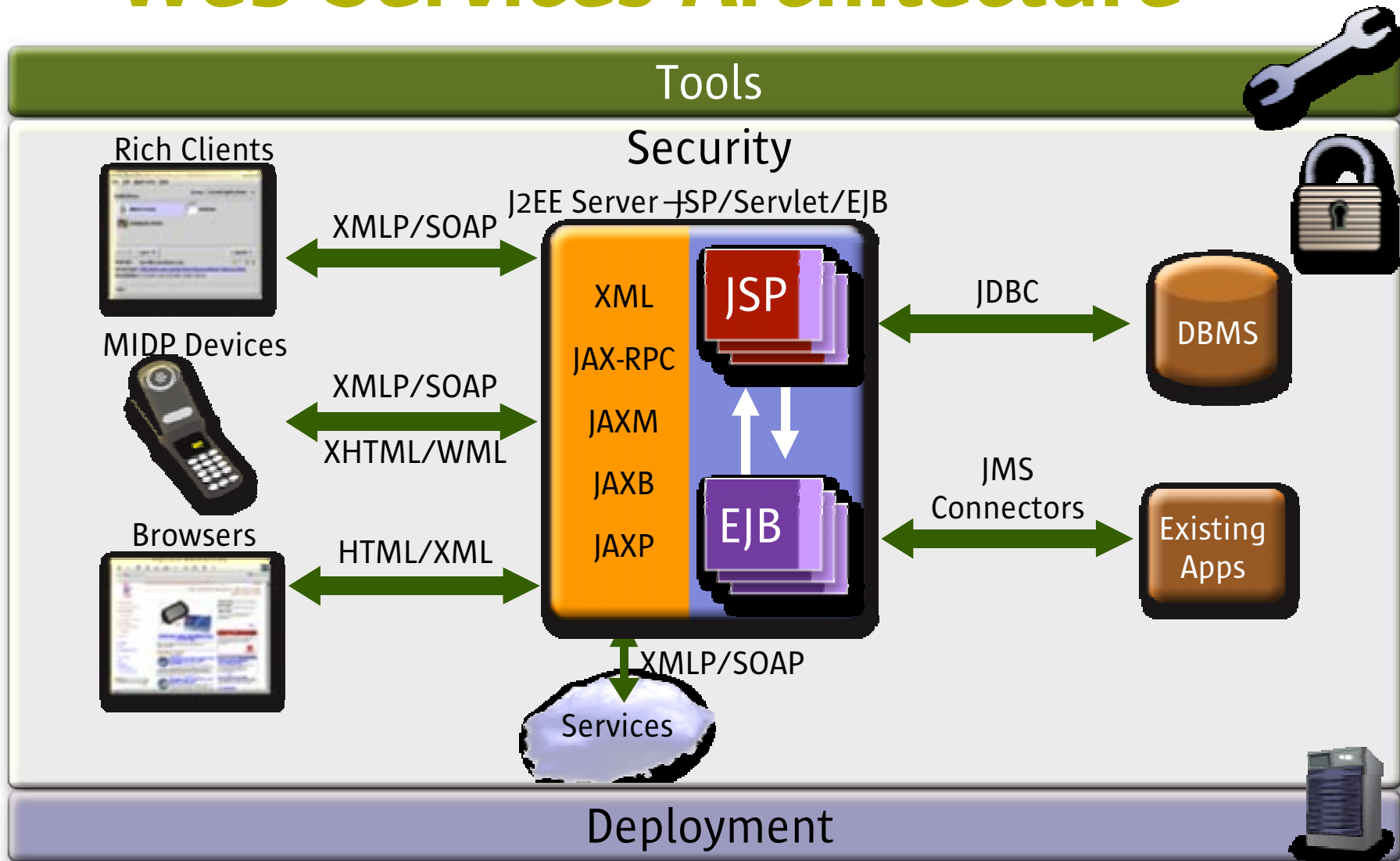




Raghavan N. Srinivas
rags@sun.com
<http://java.sun.com/security>

Technology Evangelist
Sun Microsystems

Web Services Architecture



**"Security is the chief
pretense of civilization."**

George Bernard Shaw

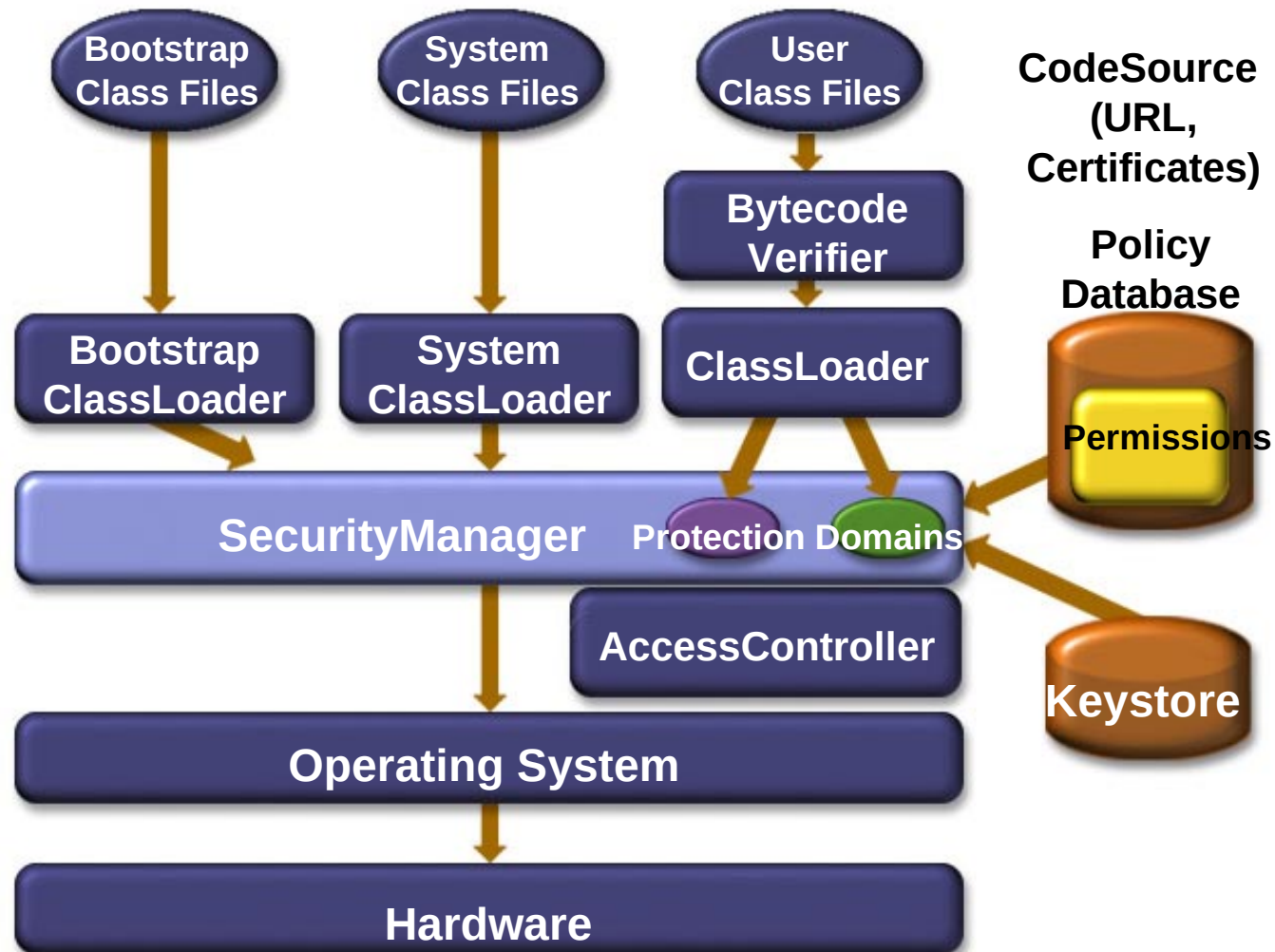
Agenda

- J2SE Security
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

JavaTM Technology-based Security (“Java Security”)

- JavaTM based software runs as designed
 - Adheres to the Java language specification and the JVMTM specification
 - Provides building blocks for secure applications
 - Resists attacks on the language and platform
 - Reduces the chance and impact of accidental programming errors

Java Security Architecture



Java Security Evolution

- Initial releases of JDK focussed on executable content threats
- Optional Packages and upcoming releases focus on distributed security threats

Java 2 Platform Security Goals

- Option of consistently treating applet and application security
- Fine-grained access control
- Well-defined Access Control Mechanism
- Concrete SecurityManager class

Calculating the Message Digest Example

```
01 import java.security.*;
02
03 // Use the SHA Algorithm
04 MessageDigest md=
05     MessageDigest.getInstance("MD5");
06 byte buf[] = Message.getBytes();
07
08 // Update the data
09 md.update(buf);
10 // After input is ready, digest the data
11 byte digestBuf[] = md.digest();
```

)

Message Digest Values

Message	MD5 Digest
I pay Bill \$1500.00	8021d0cda6ca230a5853f9d55ba4cceb
I pay Bill \$15000.00	C0239506cf350e9c6f8adcb62bd4a5c5
I pay Jill \$15000.00	752ed078a31567bf81545d21da298be9

Agenda

- J2SE Security (optional packages)
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

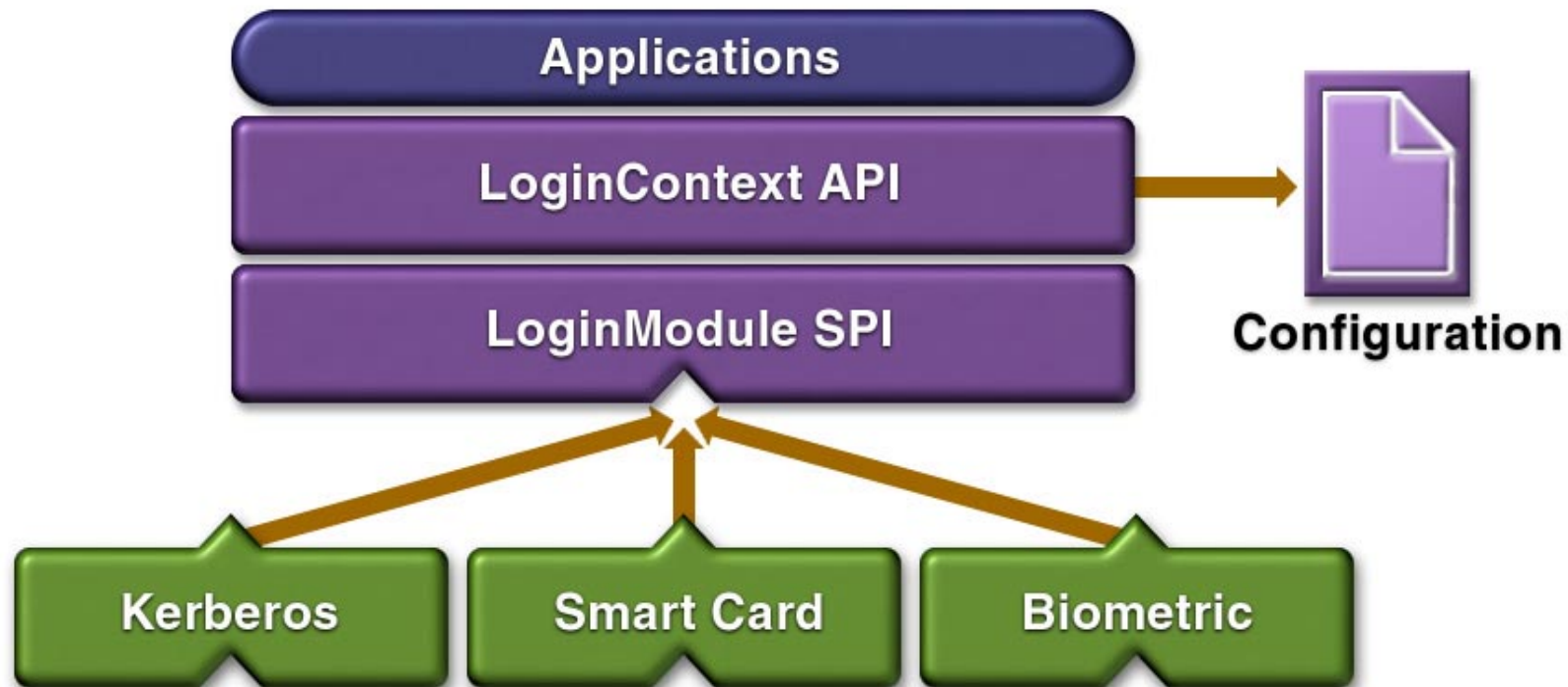
JavaTM Platform Security Extensions (optional packages)

- Java Authorization and Authentication Service API (JAAS)
- Java Cryptography Extensions (JCE)
 - Common API for applications
 - Standard SPI for security service providers
- Java Secure Socket Extension (JSSE)

JavaTM Authentication and Authorization Service (JAAS) API

- Java platform security based on
 - Where the code originated
 - Who signed the code
- The JAAS API augments this with **who's running the code**
- Pluggable authentication
- User-based authentication/authorization

JAAS Pluggable Authentication



JAAS File Entries

```
01 // Example Java 2 Security Policy Entry
02 grant Codebase "www.sun.com", Signedby "duke" {
03     FilePermission "/cdrom/-", "read";
04 }
```

```
01 // Example JAAS Security Policy Entry
02 grant Codebase "www.sun.com", Signedby "duke",
03     Principal com.sun.Principal "charlie" {
04     FilePermission "/cdrom/charlie/-", "read";
05 }
```

```
01 // Example login module configuration entry
02 Login2 {
03     sample.SampleLoginModule required;
04     com.sun.security.auth.module.NTLoginModule
                                sufficient;
05     com.foo.SmartCard requisite debug=true;
06     com.foo.Kerberos optional debug=true;
07 };
```


JAAS Programming Model

```
01 import java.security.*;
02 import javax.security.auth.*; //exts

03 // Instantiate a login context
04 LoginContext ctx = new LoginContext
    ("name", CallbackHandler);

05 // Authenticate the subject
06 ctx.login();

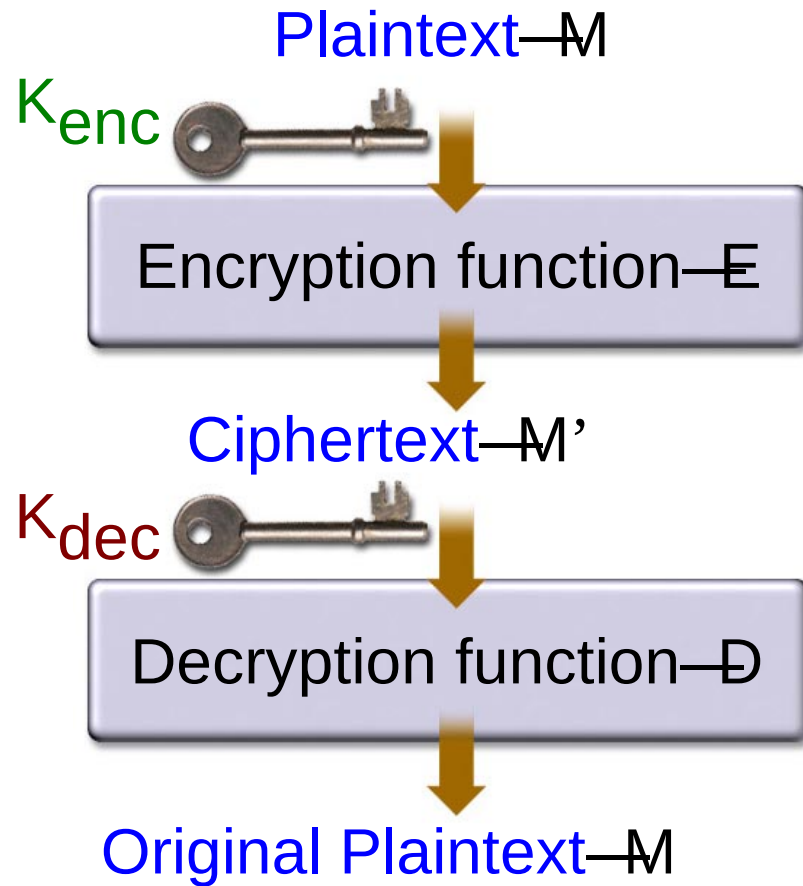
07 // Retrieve authenticated subject
08 Subject sub = ctx.getSubject();

09 // Enforce Access Controls
10 Subject.doAs(sub, action);
```

JavaTM Cryptography Extensions (JCE)

- Cryptographic APIs supplementing the Java 2 platform
- Framework for multiple CSPs
 - Comes with Sun JCE provider
 - Multiple independent providers

Cryptographic Process



M is the original message

K_{enc} is encryption key

M' is the scrambled message

K_{dec} is decryption key

From M' only, “hard” to get M

E and D are related such that

$$E(K_{enc}, M) = M'$$

$$D(K_{dec}, M') = M$$

$$D(K_{dec}, E(K_{enc}, M)) = M$$

Crypto. Processes (compared)

Public Key Cryptography

- Encryption and decryption keys are different
- Key distribution is easier
- Public key cryptography is very slow
- Examples: RSA

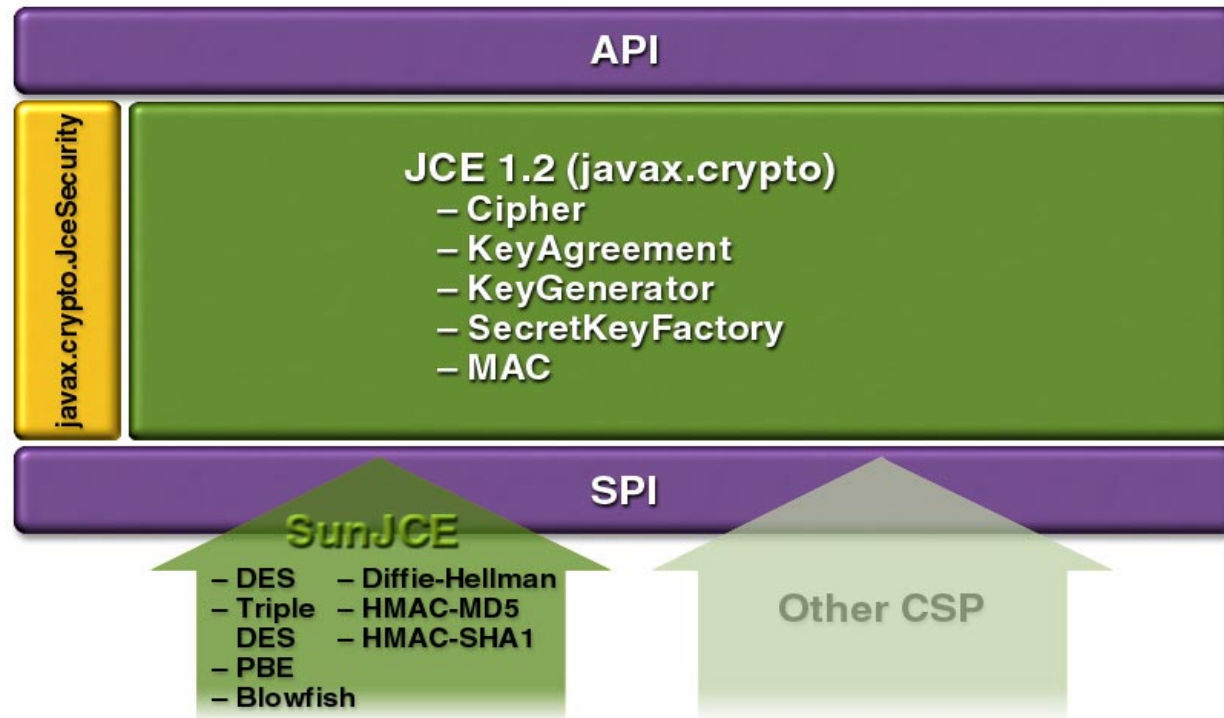
Private Key Cryptography

- Encryption and decryption keys are same
- Key distribution is an issue
- Private key cryptography is faster
- Examples: DES, AES

Session Key Cryptography

- Key negotiation and encryption separate
- Key distribution is not an issue
- Best of both approaches
- Examples: SSL

JCE 1.2 Release



JCA/JCE

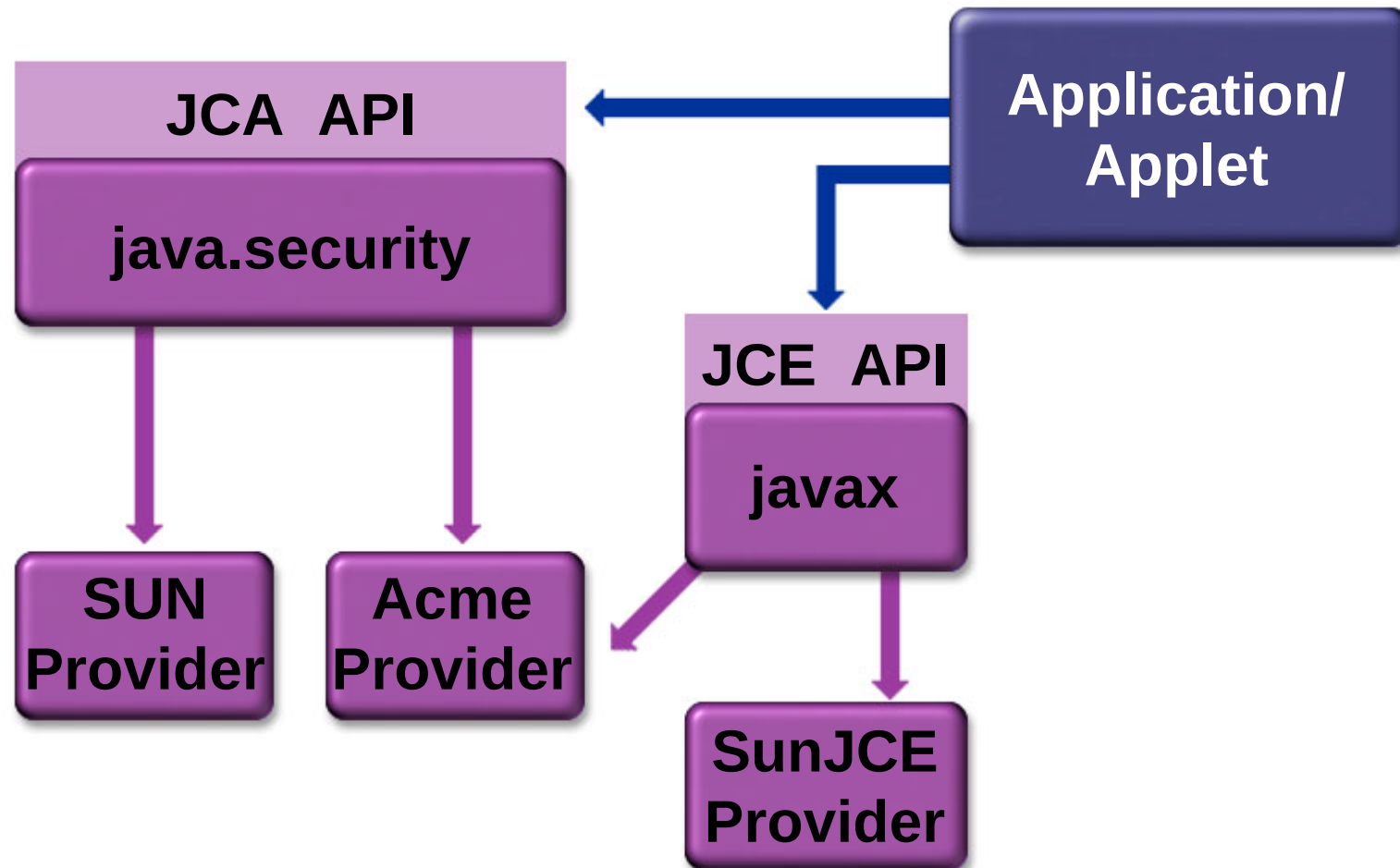
JCA

- Cryptographic Architecture
- Basic cryptography
- Export control free
- Signatures, Digests, etc.

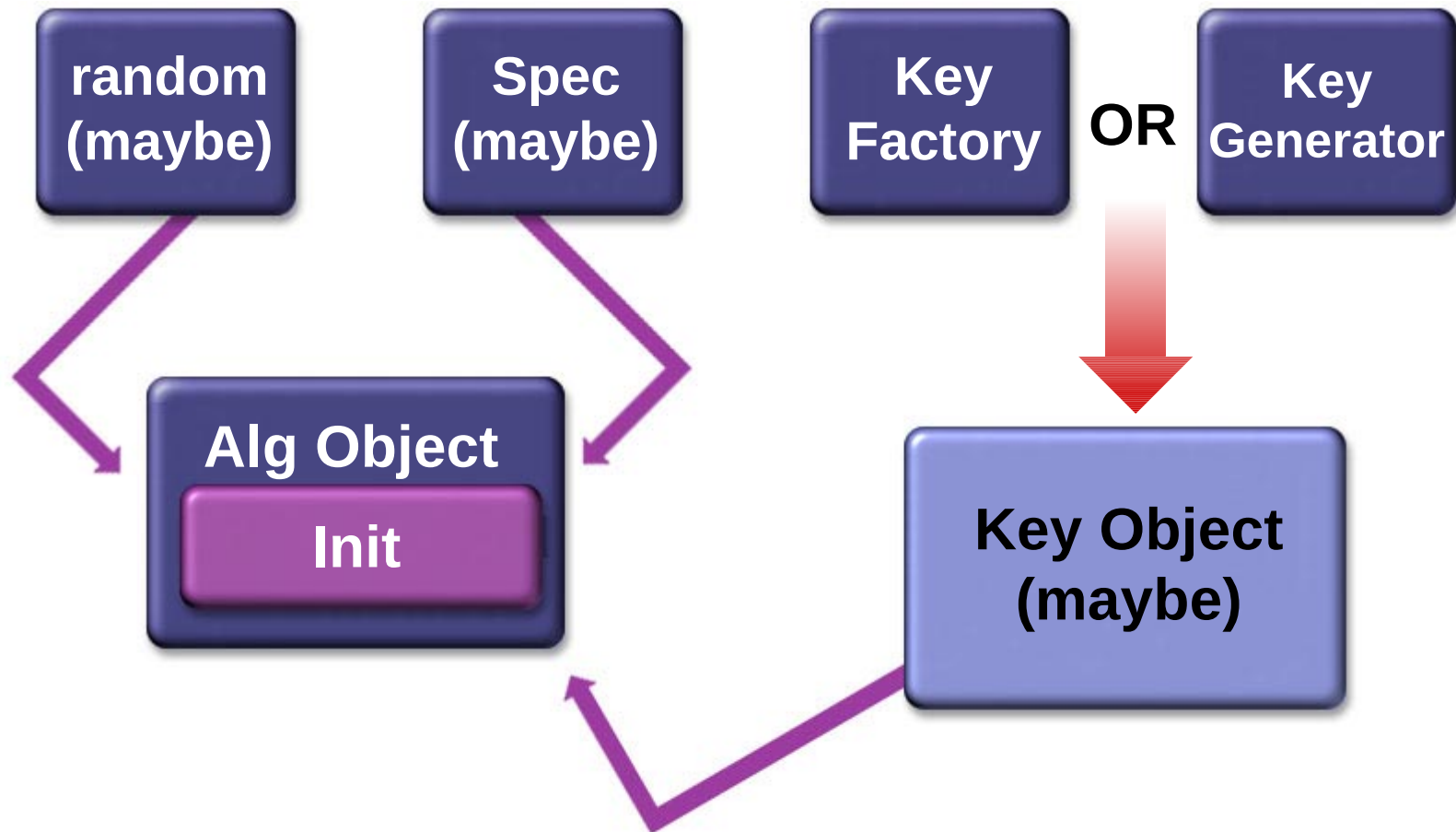
JCE

- Cryptographic Extensions
- Advanced cryptography
- Export control restrictions (originally)
- Ciphers

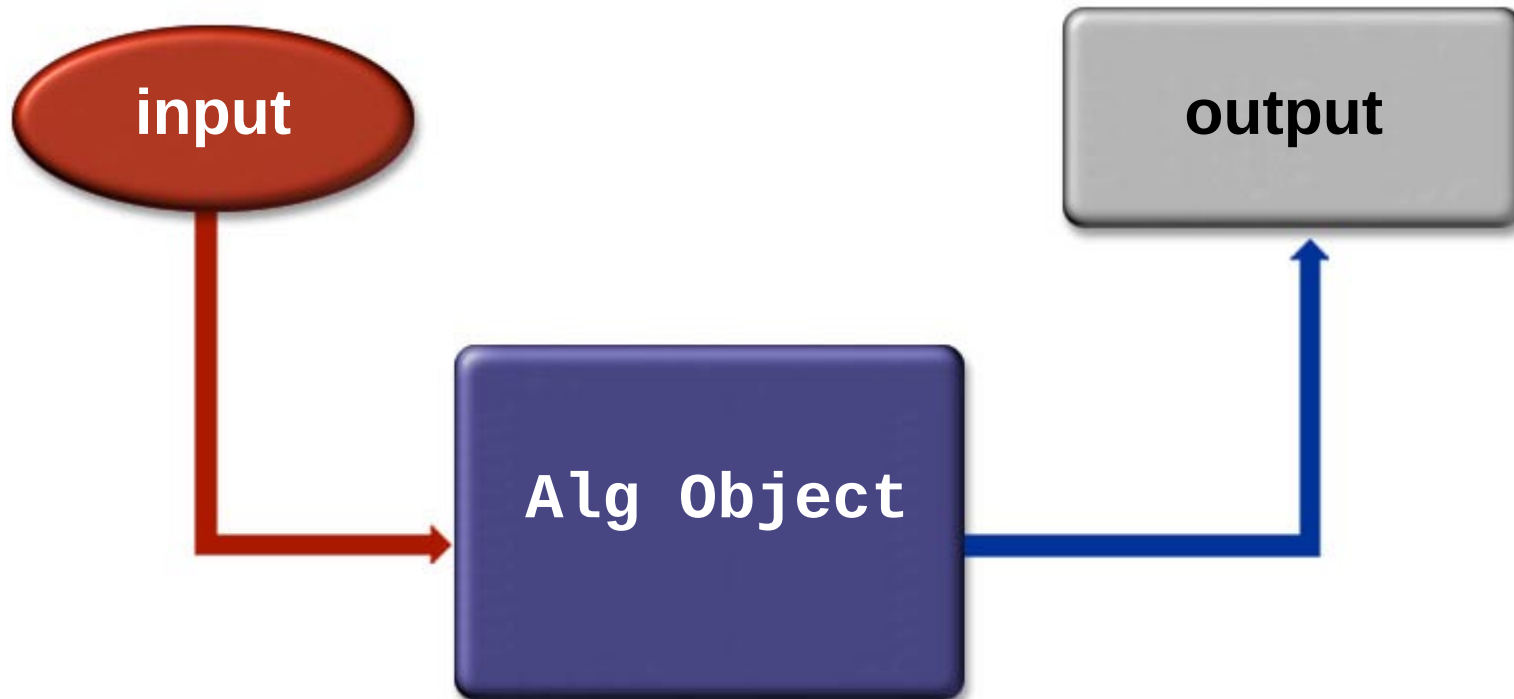
JCA/JCE



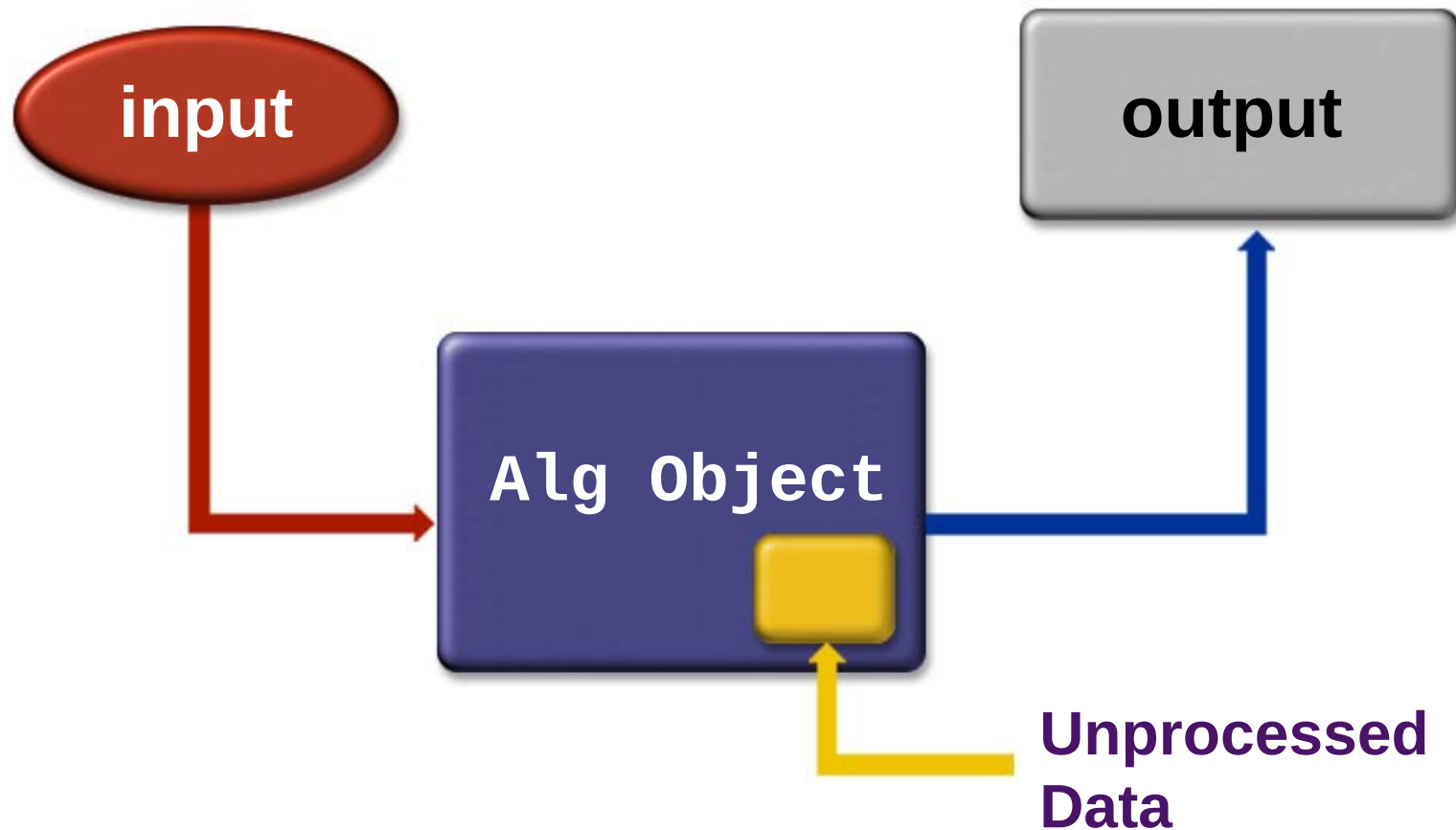
JCA/JCE Model



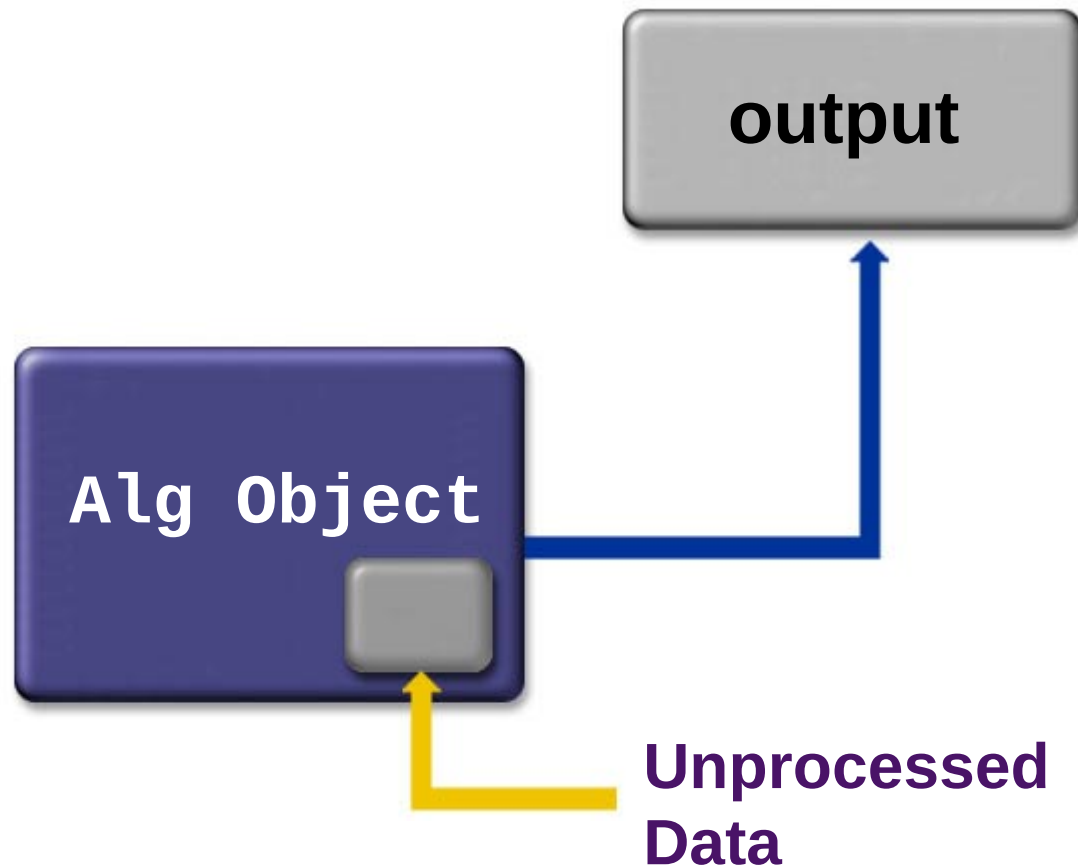
JCA/JCE Model



JCA/JCE Model



JCA/JCE Model



The JCA/JCE Model

1. **getInstance**
2. **init**
3. **update**
4. **final**

JCE Programming model

```
01 import java.security.*;
02 import javax.crypto.*;
03
04 // Get Provider
05 Provider sunJce = new
06     com.sun.crypto.provider.SunJCE();
07
08 // Obtain Cipher
09 Cipher c = Cipher.getInstance
10     ("Blowfish");
11
12 // Get the key Generator
13 KeyGenerator kgen =
14     KeyGenerator.getInstance("Blowfish");
```

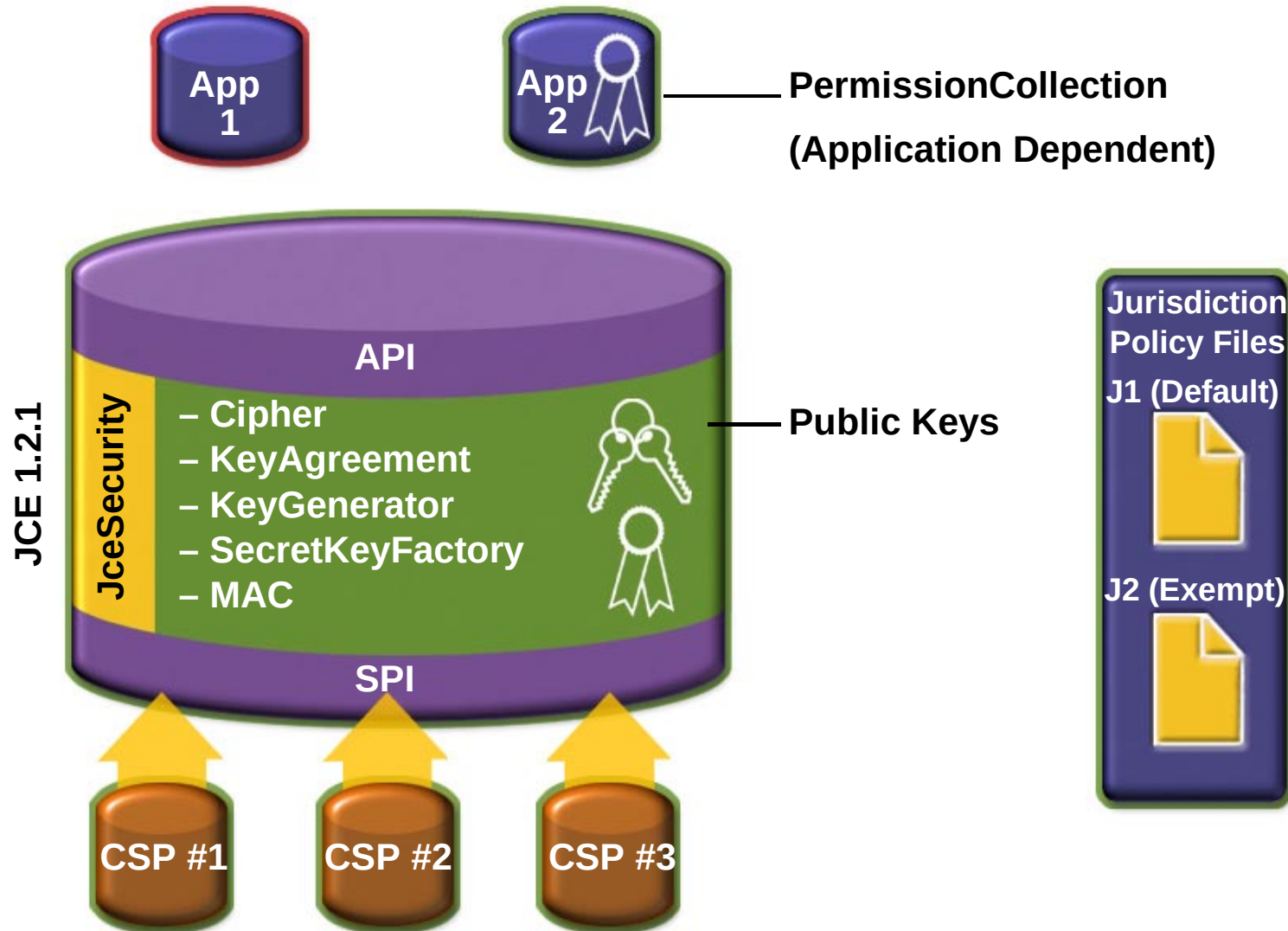
}

JCE Prog. Model (cont'd)

```
15 // Generate the key specs
16 SecretKey skey = kgen.generateKey();
17 byte[] raw = skey.getEncoded();
18 SecretKeySpec kspec = new
19     SecretKeySpec(raw, "Blowfish");
20
21 // Initialize cipher with keys, etc.
22 cipher.init(Cipher.ENCRYPT_MODE, kspec);
23
24 // Update buffers
25 while (msg[i] != null)
26     enc = cipher.update(msg[i].getBytes());
27
28 // Finish up
29 enc = cipher.doFinal();
```

)

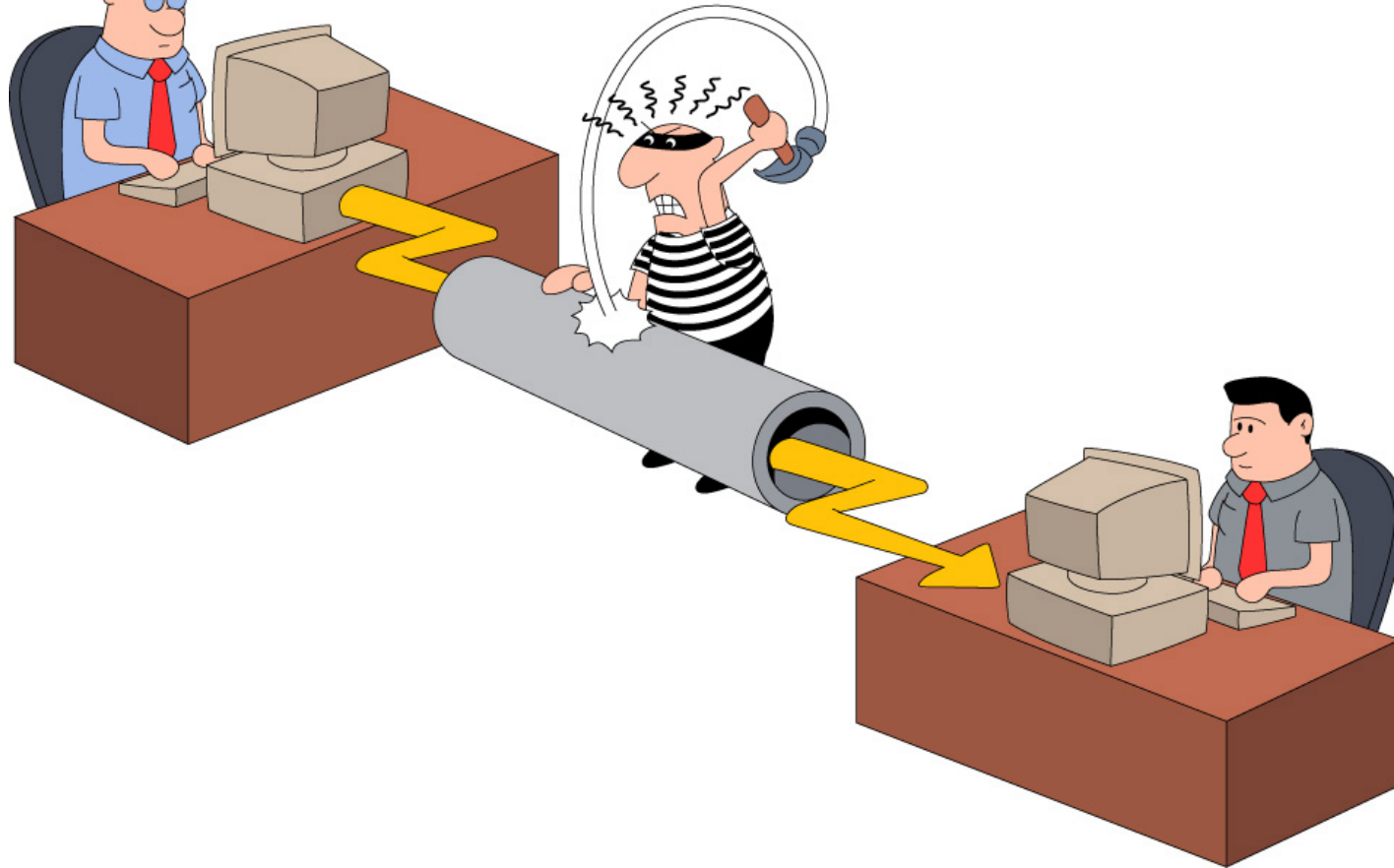
JCE 1.2.1: Architecture



JCE 1.2.1: Features

- Unapproved providers can't plug in
- Providers unusable without framework
- Crypto strength is configured in jurisdiction policy files

JavaTM Secure Socket Extension (JSSE)



What is JSSE?

- Java API for Secure Sockets Layer (SSL)
- SSL provides security at Session level or TLS
 - Confidentiality, integrity
 - Server authentication
 - Optional client authentication
- Uses algorithms, keys transparently

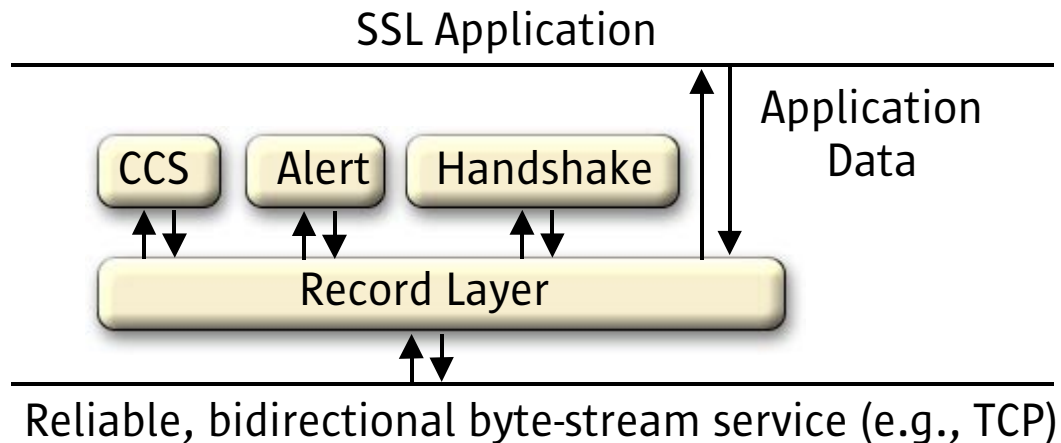
Secure Socket Layer (SSL)

- By far, the dominant security protocol on the web (https=http over ssl)
- Responsible for the emergence of e-commerce, other security sensitive services on the web
- Beneficiary of several years of public scrutiny

SSL Overview

- Operates atop bi-directional, reliable byte stream. Typically TCP
- Offers **end-to-end security** even when the underlying reliable byte stream is proxied
- Has seen several revisions

SSL's Layered Architecture



- Record layer offers bulk encryption/authentication using symmetric-key algorithms. Cleartext flow until symmetric keys are initialized
- Handshake protocol uses public-key algorithms to establish a "master-secret" used to derive MAC secrets, cipher keys/IVs

JSSE Programming model - Server SSL usage

```
01 import java.io.*;
02 import java.net.*;
03 import javax.net.ssl.*;

04 // Server
05 SSLServerSocketFactory sslsrvfact =
    SSLServerSocketFactory.getDefault();
06 SSLServerSocket s =
    sslsrvfact.createServerSocket(port);
07 s.accept();
```

```
}
```

JSSE Programming model - Client SSL usage

```
01 import java.io.*;
02 import java.net.*;
03 import javax.net.ssl.*;

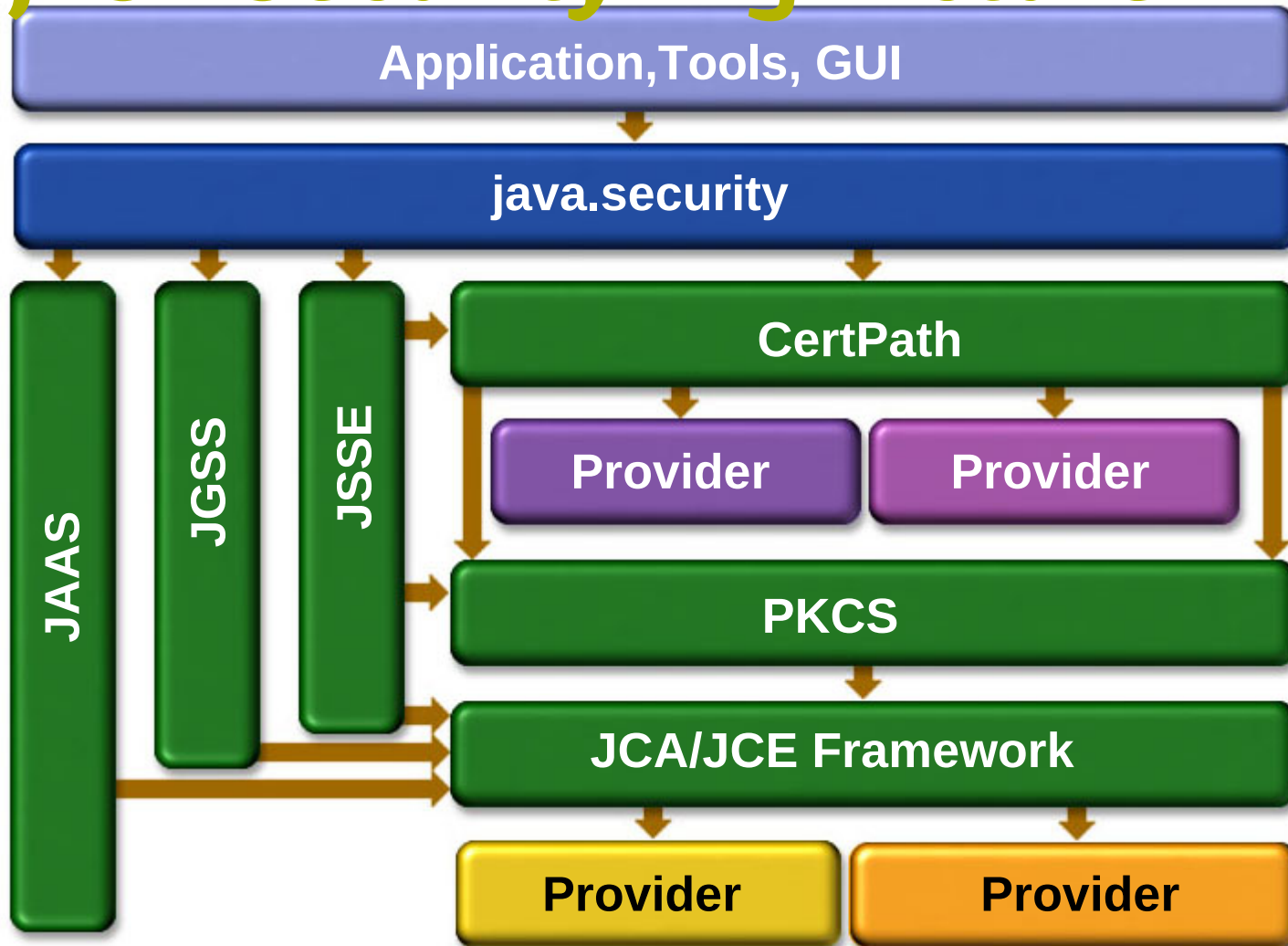
04 // Client
05 SSLSocketFactory sslfact =
    SSLSocketFactory.getDefault();
06 SSLSocket s =
    sslfact.createSocket(host, port);
```

```
)
```

J2SE Security roadmap: “Merlin” Platform

- CertPath
- Kerberos
- PKCS
- JGSS
- Optional Packages into core

Roadmap: J2SE Security Big Picture



Agenda

- J2SE Security
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

Vision

Secure

^Ubiquitous access to information.

"Anytime, Anywhere" access but

Authenticated (not "Anonymous")

Confidential (not open to just "Anyone")

Integrity-protected (not "Adulterated")

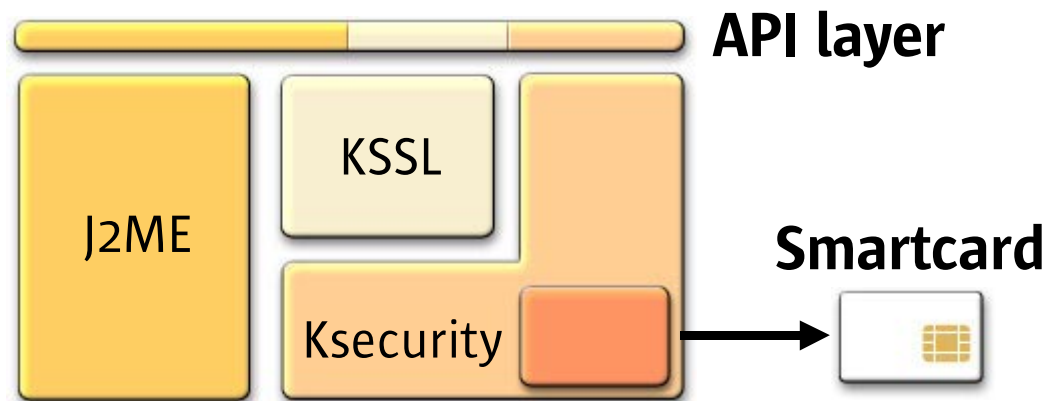
Wireless History (the W-Syndrome or W* effect)

- WAP's "wireless is different" philosophy:
UDP→WDP, TLS→WTLS, HTTP→WSP,
HTML→WML, ECMAScript→WMLScript
- No good faith feasibility study of a standards based approach

Proxy Based Architecture

- Proxy between mobile device and content server performs protocol conversions, e.g. WAP, Palm.net
- All communication forced through service provider's proxy/gateway creating a "walled garden"
- Concerns:
 - Scalability: single choke point, no end-to-end flow control
 - Legal: antitrust issues
 - Security: built-in "man-in-the-middle"

New Security Classes for J2ME



- Message digests, ciphers, random data, signatures, key modeled after JavaCard APIs.
- Certificates and certificate storage modeled after J2SE. Written in Java with some CPU intensive parts in C.

Supported Features

- **Keys:** Different lengths Symmetric keys, RSA Public/Private Keys
- **Ciphers:** ALG_RSA_PKCS1 and ALG_ARCFOUR (no DES, etc.)
- **Message Digests:** MD5 and SHA
- **Signatures:** ALG_RSA_MD5_PKCS1 and ALG_RSA_SHA_PKCS1. No DSA support.

Supported Features (Certificates and KeyStores)

- **Certificates:**
 - Only X.509 certificates containing RSA keys and signed using RSA w/ MD5 or SHA.
 - X.509v3 extensions are handled correctly.
- **KeyStore:** can only store certificates (no private keys or symmetric keys).

KSSL Features

- Client-side implementation of SSL.
Supports session reuse
- Only supports two cipher suites - RSA_RC4_128_MD5 and RSA_RC4_40_MD5 (most common and fast)
- No client-side authentication (rarely used). Server authenticated through RSA signatures
- Maintains an extensible set of trusted Certification Authorities covering > 95% of SSL server certificates [Netcraft]

J2ME security roadmap

- Presented at IETF and WAP as input for proposed merger of TLS and WTLS
- Distributed via Sun Labs technology evaluation license. HTTPS functionality in MIDP reference implementation expected mid-2001
- Demo applications available for mobile access to Sun's Intranet portal
- Exploring support for Javacards and Elliptic Curve Cryptography

J2ME Security Summary

- Security **protocols** dominant on the Internet commonly perceived as too "big" for small devices.
- Today's wireless architectures are proxy-based, no end-to-end security
- Success hinges upon **careful selection of feature subsets without destroying compatibility.**

If you ``speak'' standard protocols...



"On the Internet, nobody knows you're a dog."

... nobody knows you are a handheld (apologies to Peter Steiner)

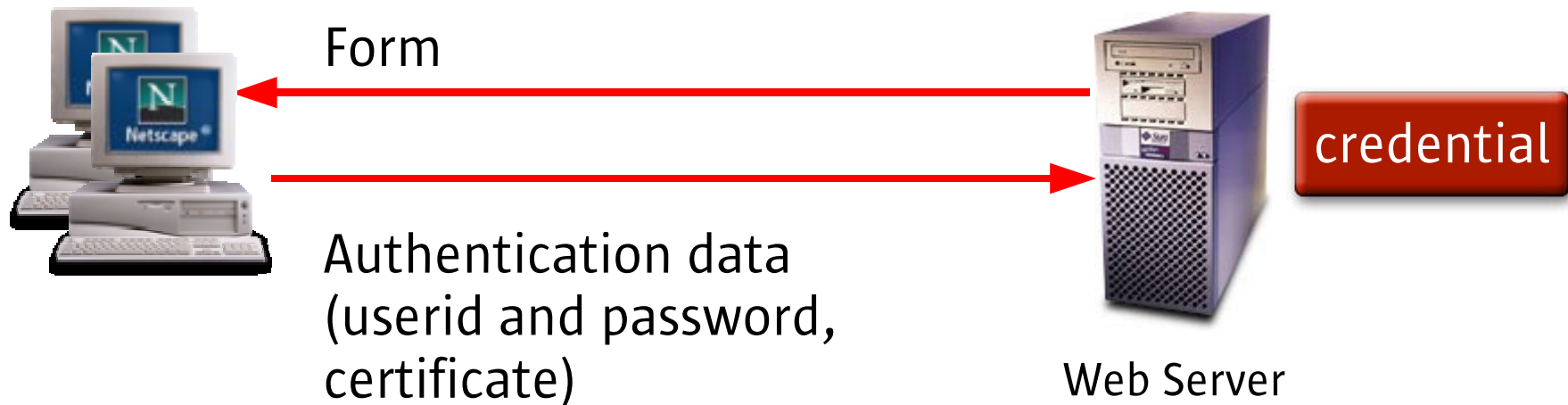
Agenda

- J2SE Security
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

Basics of J2EE™ Security

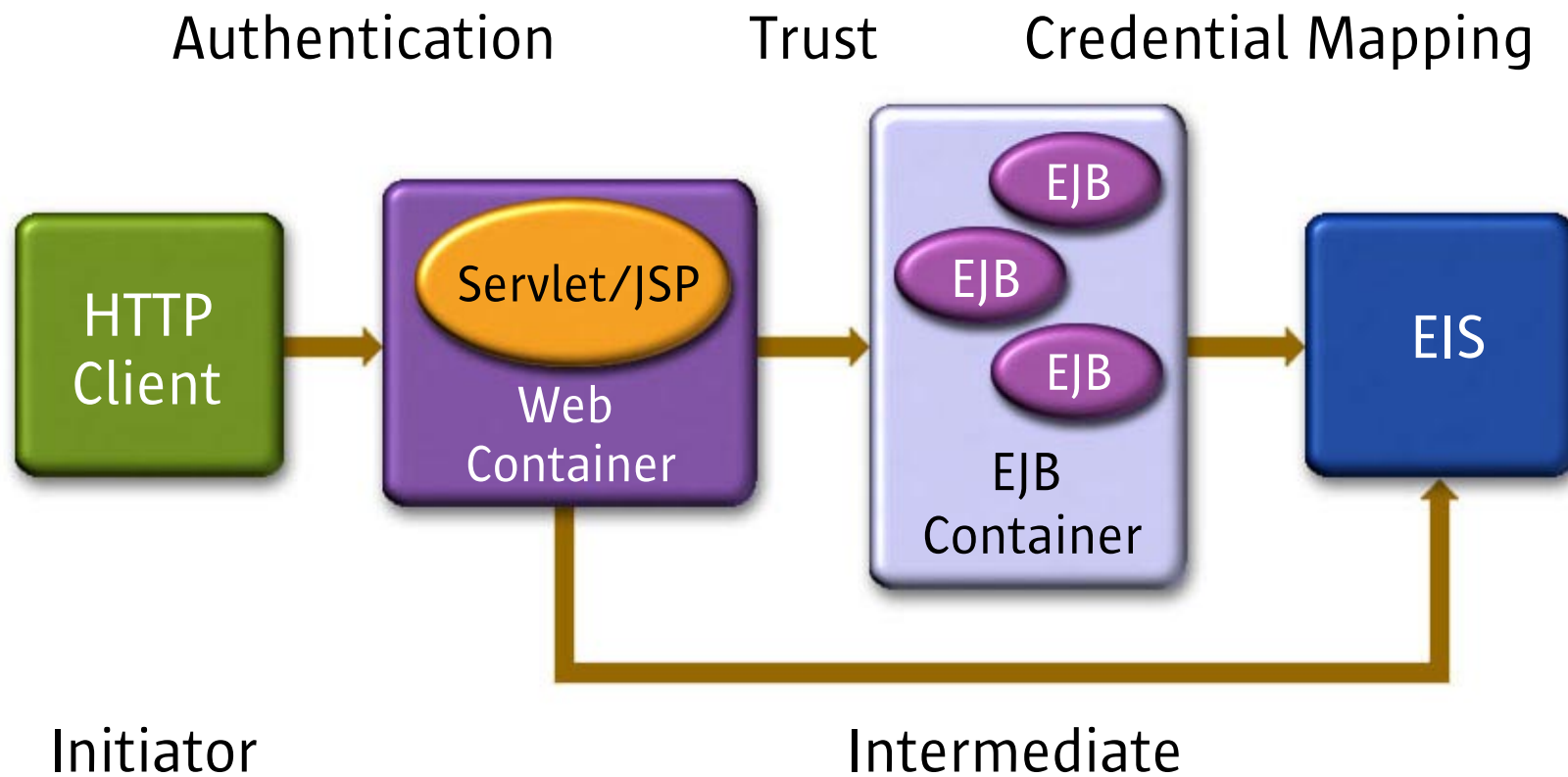
- Authentication
- Authorization:
 - Declarative
 - Programmatic or Explicit

Authentication Example



Verifies Identity

Authentication in Web Server



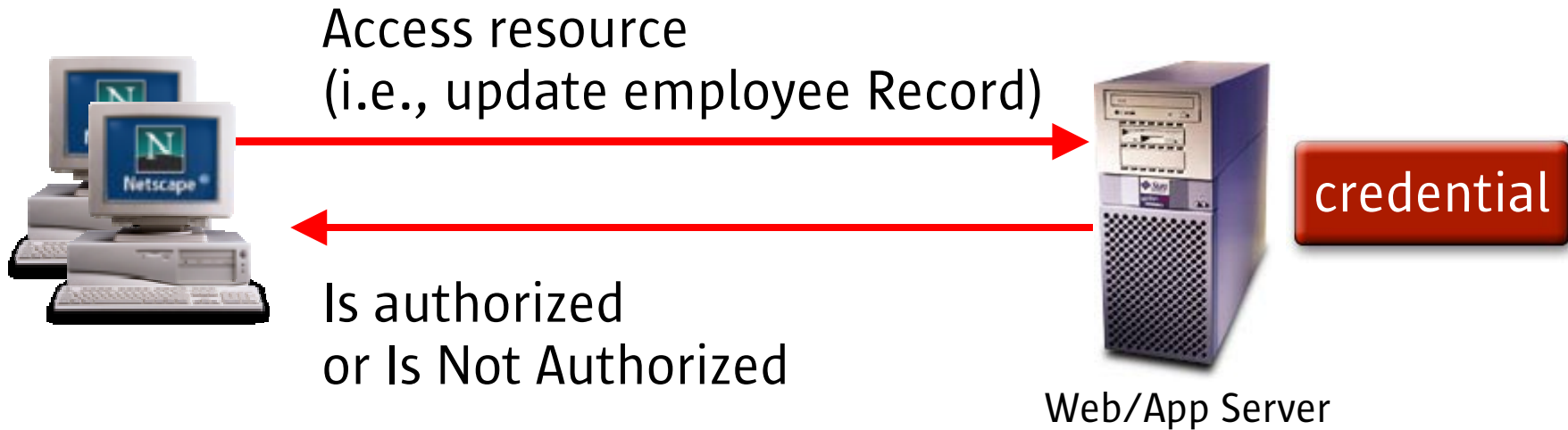
HTTP Form Authentication Method Configuration

Authentication

```
<login-config>  
  <auth-method>FORM</auth-method>  
  <form-login-page>/estore/login.jsp</form-login-page>  
  <form-error-page>/estore/error.html</form-error-page>  
</login-config>
```

other Authentication Methods configuration:
Default, and Certificate

Authentication Example



Verifies Permission

Authorization, Permissions

- **Resource protection** by **web** container
 - by **authorization–constraint**
 - Authorization–constraint** makes **authentication** a **prerequisite** for
- **Resource protection** by **EJB** container
 - by **method–permission**

Web Authorization Constraint

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>CheckOut</web-resource-name>
    <url-pattern>CheckOut</url-pattern>
    <http-method>POST</http-method>
    <http-method>GET</http-method>
  </web-resource-collection>
  <auth-constraint>
    { <role-name>customer</role-name>
  </auth-constraint>
</security-constraint>
```

authorization

)

Web Authorization and Authentication

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>checkout</web-resource-name>
    <url-pattern>Checkout</url-pattern>
    <http-method>POST</http-method>
    <http-method>GET</http-method>
  </web-resource-collection>
```

Authorization

```
    <auth-constraint>
      <role-name>customer</role-name>
    </auth-constraint>
  </security-constraint>
```

Authentication

```
<login-config>
  <auth-method>FORM</auth-method>
  <form-login-page>/estore/login.jsp</form-login-page>
  <form-error-page>/estore/error.html</form-error-page>
</login-config>
```

Login Form

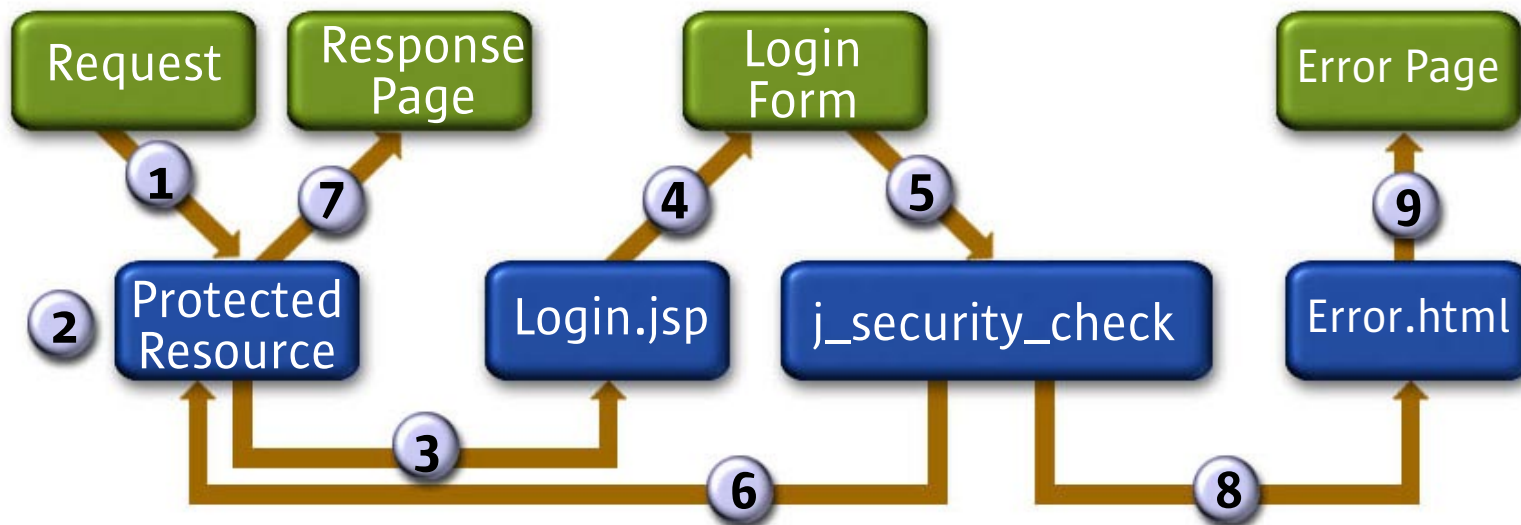
- **Form-based login**

- Login.jsp identified in deployment descriptor contains:

```
<form method="POST" action="j_security_check">  
<input type="text" name="J_USERNAME">  
<input type="password" name="J_PASSWORD">  
</form>
```

- **This form must be in the web .war**

Form-based Login in Detail



1. Request made by client

2. Is client authentic?

3. Unauthenticated client
redirected

4. Login form returned to client

5. Client submits login form

6. **Authentication** Login succeeded,
redirected to resource

7. **Authorization** Permission tested,
result returned

8. Login failed, redirect to error page

9. Error page returned to client

EJB Authorization

⑤ Declarative:

- Method permissions and security roles declared in deployment descriptor, container performs authorization checks.

⑤ Programmatic:

- Bean provider uses `EJBContext isCallerInRole()` to check permissions and and `getCallerPrincipal()` to check identity .

Security Roles

what types of users will access the bean?

§ *customers can make Orders*

§ *Special Customers can Browse Discounts in Catalog*

§ *Each of these user categories is called a **ROLE**.*

<assembly-descriptor>

<security-role>

<description>May place orders, complete purchases</description>

<role-name>Customer</role-name>

</security-role>

<security-role>

<description>May Browse and purchase Discounted Items</description>

<role-name>Gold_Customer</role-name>

</security-role>

Method Permissions

Method **permissions** indicate **which roles** are allowed to **invoke** which **methods**.

<method-permission>

<role-name>Customer**</role-name>**

<method>

<ejb-name>Order**</ejb-name>**

<method-name>ejbCreate**</method-name>**

</method>

</method-permission>

<method-permission>

<role-name>Gold_Customer**</role-name>**

<method>

<ejb-name>Catalog**</ejb-name>**

<method-name>Browse_Discounts**</method-name>**

</method>

</method-permission>

Deployer Map Roles to Users/Groups

Map the **roles** to the actual **users** (or **groups**) of the operating environment

```
<rolemapping>
```

```
  <role name="Customer">
```

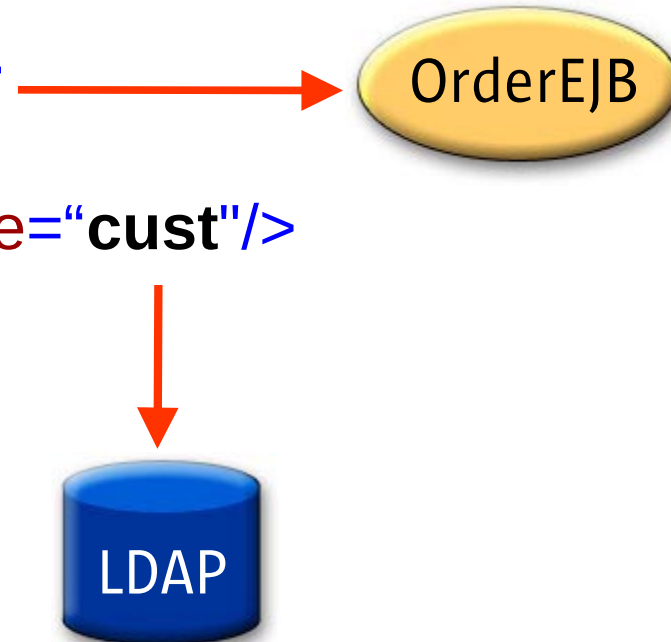
```
    <groups>
```

```
      <group name="cust"/>
```

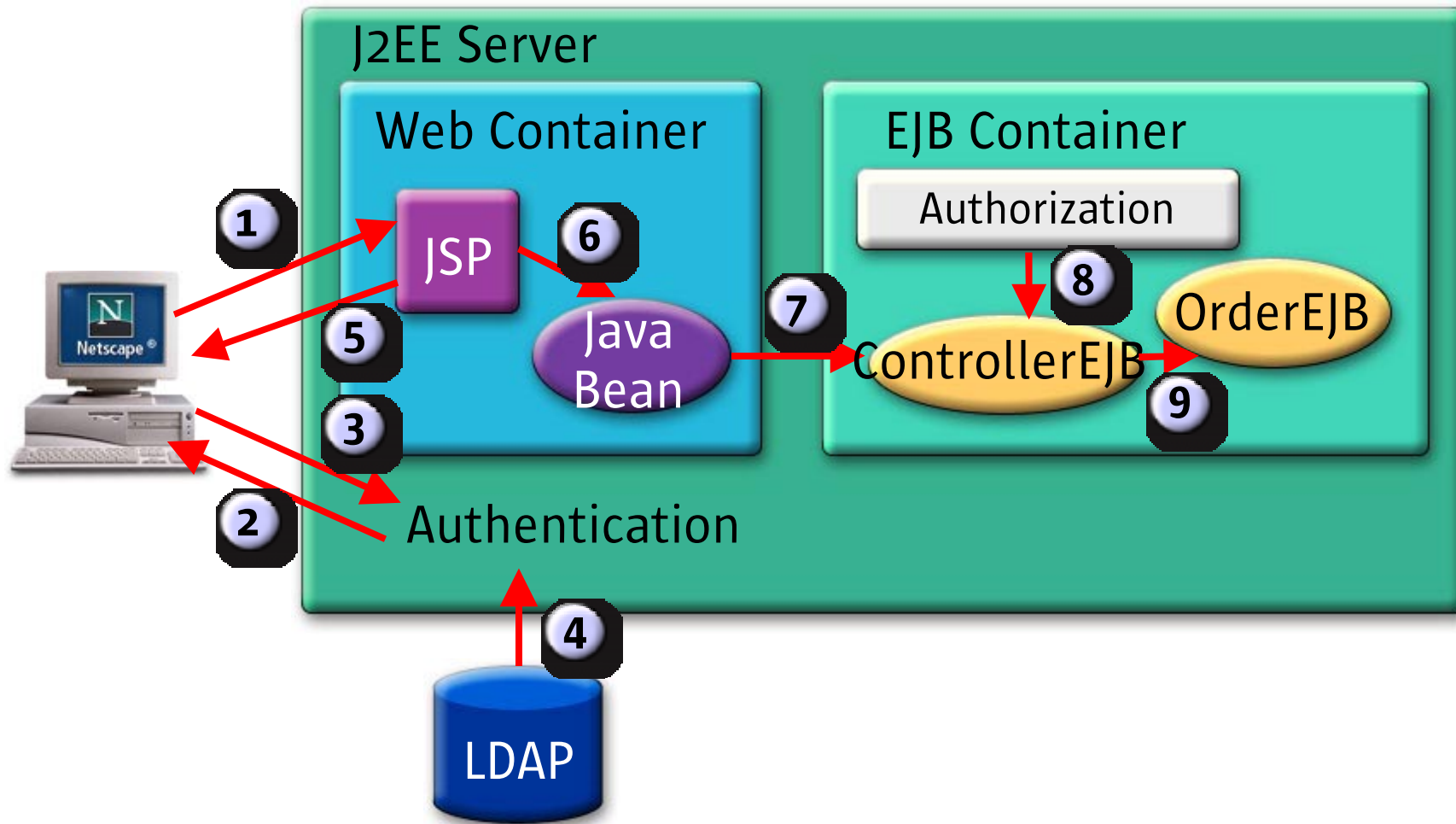
```
    </groups>
```

```
  </role>
```

```
</rolemapping>
```



Authenticated Access to JSP and EJB



Programmatic Security

- ⑤ For more complex security checks, use programmatic APIs
- ⑤ Servlets - `HttpServletRequest` interface has `getRemoteUser`, `getUserPrincipal` and `isUserInRole` methods
- ⑤ EJBs – `getCallerPrincipal.getName()` and `isCallerInRole` in `EJBContext`

Example Programmatic Security

```
public double getSalary(int employeeId) {  
  
    String callerId = ctx.getCallerPrincipal.getName();  
    String callerRole = ctx.isCallerInRole();  
  
    if ( (callerId == employeeId) &&  
        (callerRole == "employee") ) {  
        // allow access  
        return super.getSalary(employeeId);  
    } else {  
        throw new SecurityException("access denied");  
    }  
}
```

“Employees” can only access their own Salary

)

Determination of Caller Identity

- Web components
 - Get Representation of User Identity as String (also tell if authenticated user)
- EJB components
 - Get Representation of Caller Identity as String
- Portability Implications

`HttpServletRequest.getUserPrincipal().getName()`
`EJBContext.getCallerPrincipal().getName()`

Mechanisms: Caller in Role Membership

- Web and EJB components
 - Test Caller for Membership in Specified Role
 - Declarative Aspect
 - Linkage of Application Embedded Role Identifiers to Runtime Roles

```
HttpServletRequest.isUserInRole("doctor")  
)
```

```
EJBContext.isCallerInRole("doctor")
```

```
<security-role-ref>
```

```
    <role-name> doctor </role-name>
```

```
    <role-link> physician </role-link>
```

```
</security-role-ref>
```


Servlet/JSP Security

- Servlets 2.2
 - Request Attributes for
 - Certificates used for authentication
- Servlets 2.3
 - SSL requests: attributes for
 - Encryption level
 - Cipher suite

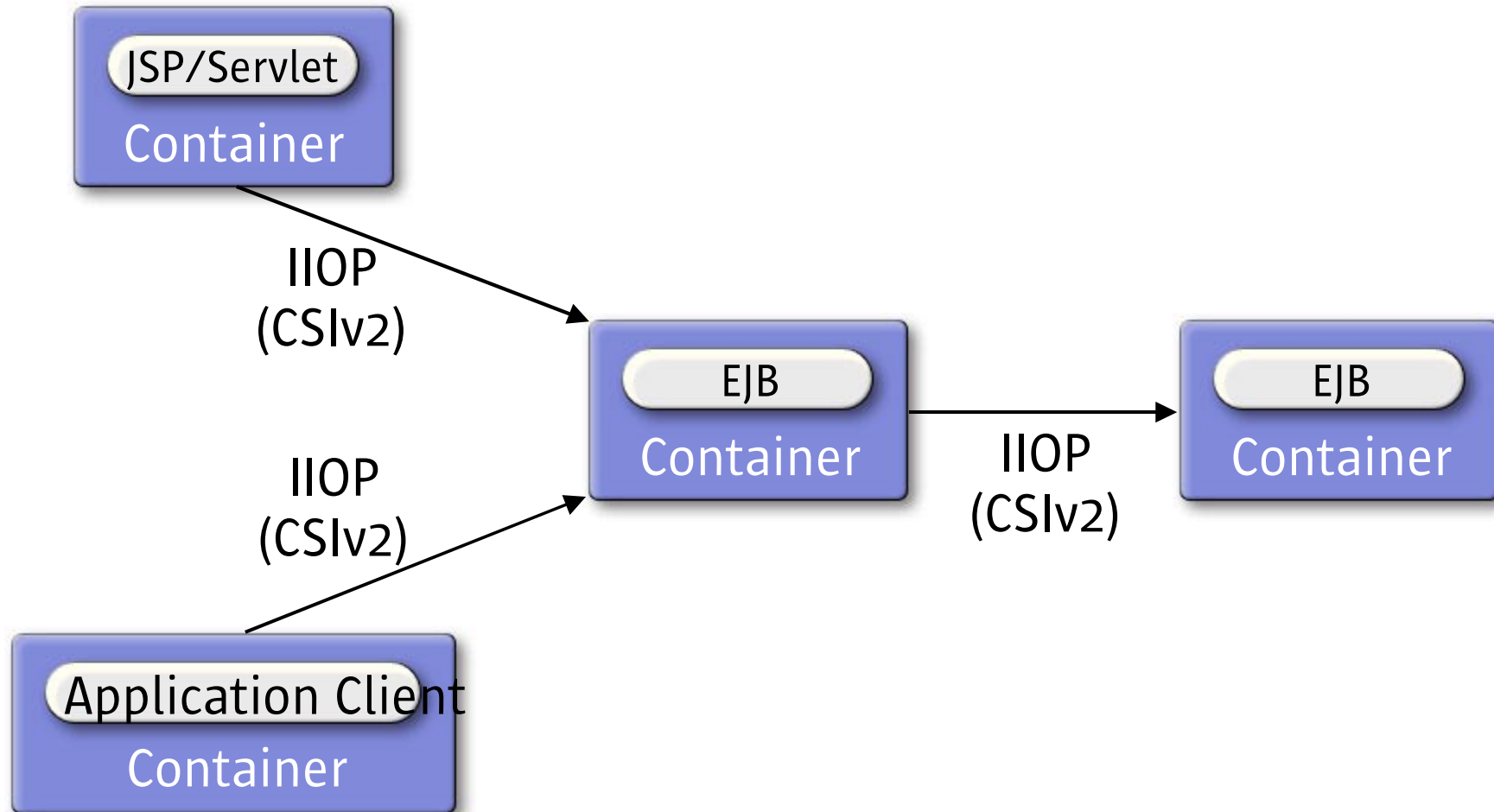
J2EE 1.3 Security Features

- EJB 2.0 Secure Interoperability
- Declarative Security Enhancements
- Connector Security

EJB 2.0 Secure Interoperability

- CSlv2 Level 0 compliance
- Reference Implementation
 - implements CSlv2 Level 0
- Conformance Test Suite
 - verifies on-the-wire interoperability
 - IOR generation
 - protocol state machine
 - token formats

EJB 2.0 Security Interoperability



J2EE Security Summary

You should now understand the basics for J2EE Security:

- Authentication
- Authorization:
 - ⊗ Declarative
 - ⊗ Programmatic or Explicit

Agenda

- J2SE Security
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

XML Security Efforts

- XML Digital Signatures
- XML Encryption
- XKMS: XML Key Management Services
- SAML: Security Assertion Markup Language
- XACML: eXtensible Access Control Markup Language

XML Digital Signatures

- Joint W3C/IETF Working Group
 - Procedures for computing/verifying signatures
 - Canonicalization of data
 - Trust in key is out-of-scope
- Data authentication, integrity & non-repudiation
- Specs: W3C Candidate Rec., RFC 3075

Example of Signed Purchase Order

```
<Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
  <SignedInfo>
    <CanonicalizationMethod Algorithm="http://www.w3.org/TR/2000/..." />
    <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"/>
    <Reference URI="#PurchaseOrder">
      <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" />
      <DigestValue>qZk+nkcGcWq6piVxeFdcBJzQ2J0=</DigestValue>
    </Reference></SignedInfo>
    <SignatureValue>IWijxQjUrcXBYc0ei4QxjWo9Kg8Dep9tlWoT4
      SdeRT87GH03dgh</SignatureValue>
  <KeyInfo>
    <X509Data>
      <X509SubjectName>CN=Alice Smith, STREET=742 Park Avenue,
        L=New York, ST=NY, C=US</X509SubjectName>
    </X509Data>
  </KeyInfo>
</Signature>
```

Example of Signed Purchase Order

```
<Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
  <SignedInfo>
    <CanonicalizationMethod Algorithm="http://www.w3.org/TR/2000/..." />
    <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"/>
    <Reference URI="#PurchaseOrder">
      <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" />
      <DigestValue>qZk+nkcGcWq6piVxeFdcBJzQ2J0=</DigestValue>
    </Reference></SignedInfo>
    <SignatureValue>IWijxQjUrcXBYc0ei4QxjWo9Kg8Dep9t lWoT4
      SdeRT87GH03dgh</SignatureValue>
  <KeyInfo>
    <X509Data>
      <X509SubjectName>CN=Alice Smith, STREET=742 Park Avenue,
        L=New York, ST=NY, C=US</X509SubjectName>
    </X509Data>
  </KeyInfo>
</Signature>
```

Example of Signed Purchase Order

```
<Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
  <SignedInfo>
    <CanonicalizationMethod Algorithm="http://www.w3.org/TR/2000/..." />
    <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"/>
    <Reference URI="#PurchaseOrder">
      <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" />
      <DigestValue>qZk+nkcGcWq6piVxeFdcBJzQ2J0=</DigestValue>
    </Reference></SignedInfo>
    <SignatureValue>IWijxQjUrcXBYc0ei4QxjWo9Kg8Dep9tlWoT4
      SdeRT87GH03dgh</SignatureValue>
  <KeyInfo>
    <X509Data>
      <X509SubjectName>CN=Alice Smith, STREET=742 Park Avenue,
        L=New York, ST=NY, C=US</X509SubjectName>
    </X509Data>
  </KeyInfo>
</Signature>
```

XML Encryption

- W3C Working Group
 - XML syntax for encrypted data
 - Encrypting/decrypting such data
 - Trust in key is out-of-scope
 - Leverages XML Digital Signatures
- Data privacy
- Working draft of requirements available

Example of Encryption

```
<purchaseOrder>
  <name>Alice Smith</name>
  <address> ... </address>
  <EncryptedData xmlns='http://www.w3.org/2000/11/temp-xmlenc'>
    <EncryptionMethod Algorithm="urn:nist-gov:tripledes-edc-cbc">
      <s0:IV xmlns:s0='http://somens'>ABCD</s0:IV>
    </EncryptionMethod>
    <KeyInfo xmlns='http://www.w3.org/2000/09/xmlsig#'>
      <KeyName>SharedKey</KeyName>
    </KeyInfo>
    <CipherData>A23B45C56</CipherData>
  </EncryptedData>
  <prodNumber>8a32gh19908</prodNumber>
  <quantity>1</quantity>
</purchaseOrder>
```

Example of Encryption

```
<purchaseOrder>
  <name>Alice Smith</name>
  <address> ... </address>
  <EncryptedData xmlns='http://www.w3.org/2000/11/temp-xmlenc'>
    <EncryptionMethod Algorithm="urn:nist-gov:tripledes-edc-cbc">
      <s0:IV xmlns:s0='http://somens'>ABCD</s0:IV>
    </EncryptionMethod>
    <KeyInfo xmlns='http://www.w3.org/2000/09/xmlsig#'>
      <KeyName>SharedKey</KeyName>
    </KeyInfo>
    <CipherData>A23B45C56</CipherData>
  </EncryptedData>
  <prodNumber>8a32gh19908</prodNumber>
  <quantity>1</quantity>
</purchaseOrder>
```

Example of Encryption

```
<purchaseOrder>
  <name>Alice Smith</name>
  <address> ... </address>
  <EncryptedData xmlns='http://www.w3.org/2000/11/temp-xmlenc'>
    <EncryptionMethod Algorithm="urn:nist-gov:tripledes-edc-cbc">
      <s0:IV xmlns:s0='http://somens'>ABCD</s0:IV>
    </EncryptionMethod>
    <KeyInfo xmlns='http://www.w3.org/2000/09/xmlsig#'>
      <KeyName>SharedKey</KeyName>
    </KeyInfo>
    <CipherData>A23B45C56</CipherData>
  </EncryptedData>
  <prodNumber>8a32gh19908</prodNumber>
  <quantity>1</quantity>
</purchaseOrder>
```

Example of Encryption

```
<purchaseOrder>
  <name>Alice Smith</name>
  <address> ... </address>
  <EncryptedData xmlns='http://www.w3.org/2000/11/temp-xmlenc'>
    <EncryptionMethod Algorithm="urn:nist-gov:tripledes-edc-cbc">
      <s0:IV xmlns:s0='http://somens'>ABCD</s0:IV>
    </EncryptionMethod>
    <KeyInfo xmlns='http://www.w3.org/2000/09/xmldsig#'>
      <KeyName>SharedKey</KeyName>
    </KeyInfo>
    <CipherData>A23B45C56</CipherData>
  </EncryptedData>
  <prodNumber>8a32gh19908</prodNumber>
  <quantity>1</quantity>
</purchaseOrder>
```


XML Key Management System (XKMS)

- Submitted as a W3C comment
- Two parts:
 - XKISS: XML Key Information Service Spec.
 - A protocol for a trust service for resolving public keys in XML Sig and Enc
 - Moves PKI complexity from client to server
 - Lowers PKI overhead for PDAs, mobile phones, etc.
 - Not PKI dependent on X.509, PGP, etc

XKMS (continued)

- 2nd part:
 - XKRSS: XML Key Registration Service Spec
 - A protocol for a web service that accepts registration of public keys
 - Tiered service model

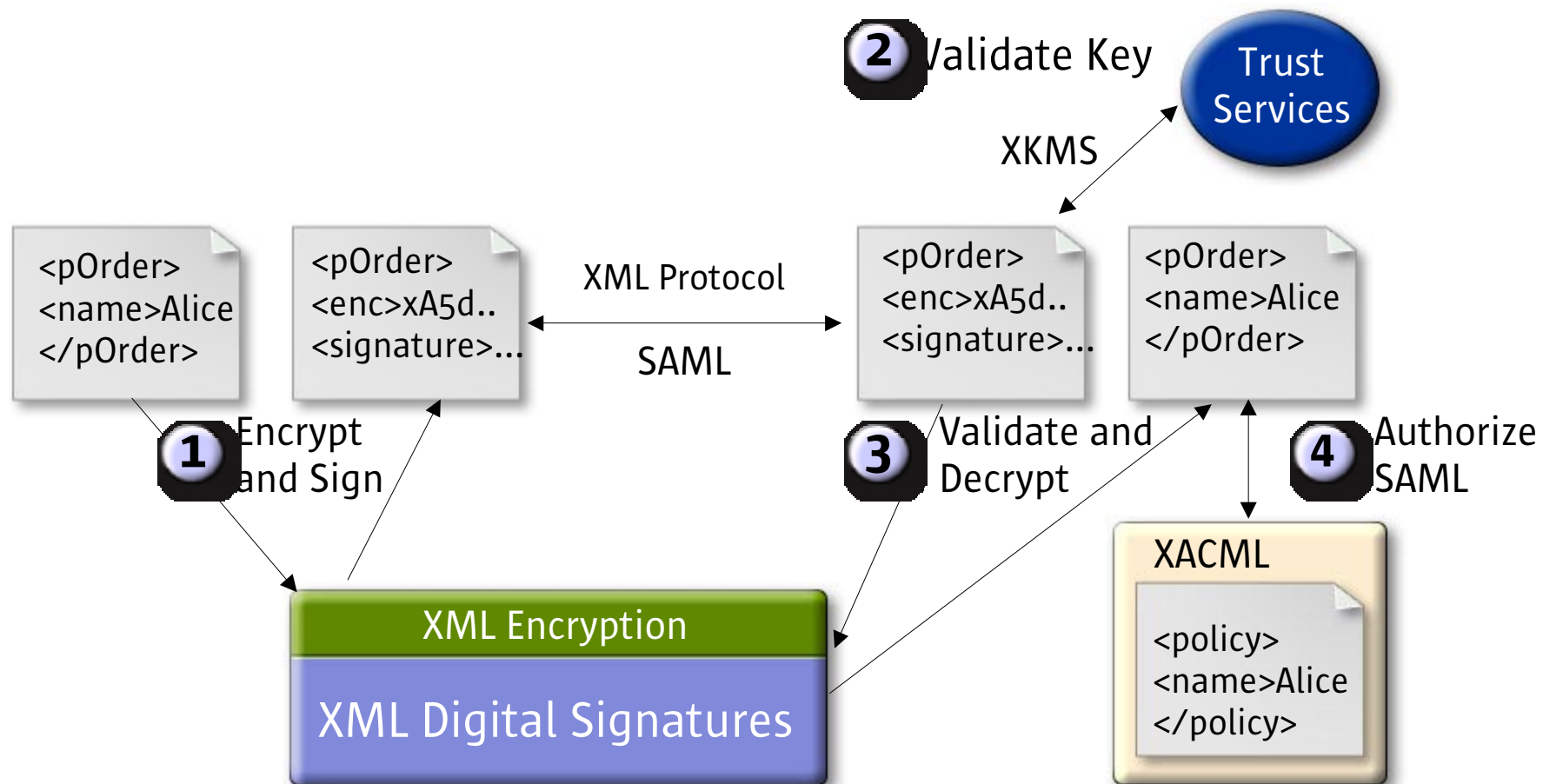
Security Assertions Markup Language (SAML)

- OASIS security-services TC
- Define an XML framework for exchanging authentication and authorization info
 - Various XML security assertions: credentials, authentication, attribute, etc...
 - Request & response protocol
 - SSO for B2B applications

eXtensible Access Control Markup Language (XACML)

- Define a core schema and namespace for authorization policies in XML:
 - Against XML objects
 - Extensible
- OASIS Tech Committee formed in April
- Closely aligned with SAML effort

Architectural View



XML Policy Provider Project

- DTD/Schema for Java Policy
- Java Policy in LDAP Directory
- Enhance Java Policy Provider implementation
- Protect with XML Digital Signature
- Use XACML schema

XML Security Summary

- Many new XML security standards being developed
- JSRs 104-106 in development through JCP
- Provide foundation for building a secure solution with XML and Java using J2SE

J2SE and XML and Security (in planning stages)

- JSR 105: XML Digital Signature API
- JSR 106: XML Encryption API
- JSR 104: Trust Services (XKMS)
- XML Policy Provider

J2EE Security Resources

- ⑤ J2EE 1.2 specification, Servlet Specification, EJB Specification, Blueprints
<http://java.sun.com/j2ee>
- ⑤ Wiley: Mastering Enterprise JavaBeans By Ed Roman <http://www.theServerSide.com>
- ⑤ <http://www.sun.com/developer> to download code camps (under training)
- ⑤ <http://archives.java.sun.com> to ask questions in discussion groups about j2ee or ejb

XML security Resources

XML Dsig: <http://www.w3.org/Signature>

XML Enc: <http://www.w3.org/Encryption/2001/>

XKMS: <http://www.w3.org/TR/xkms/>

SAML: <http://oasis-open.org/committees/security>

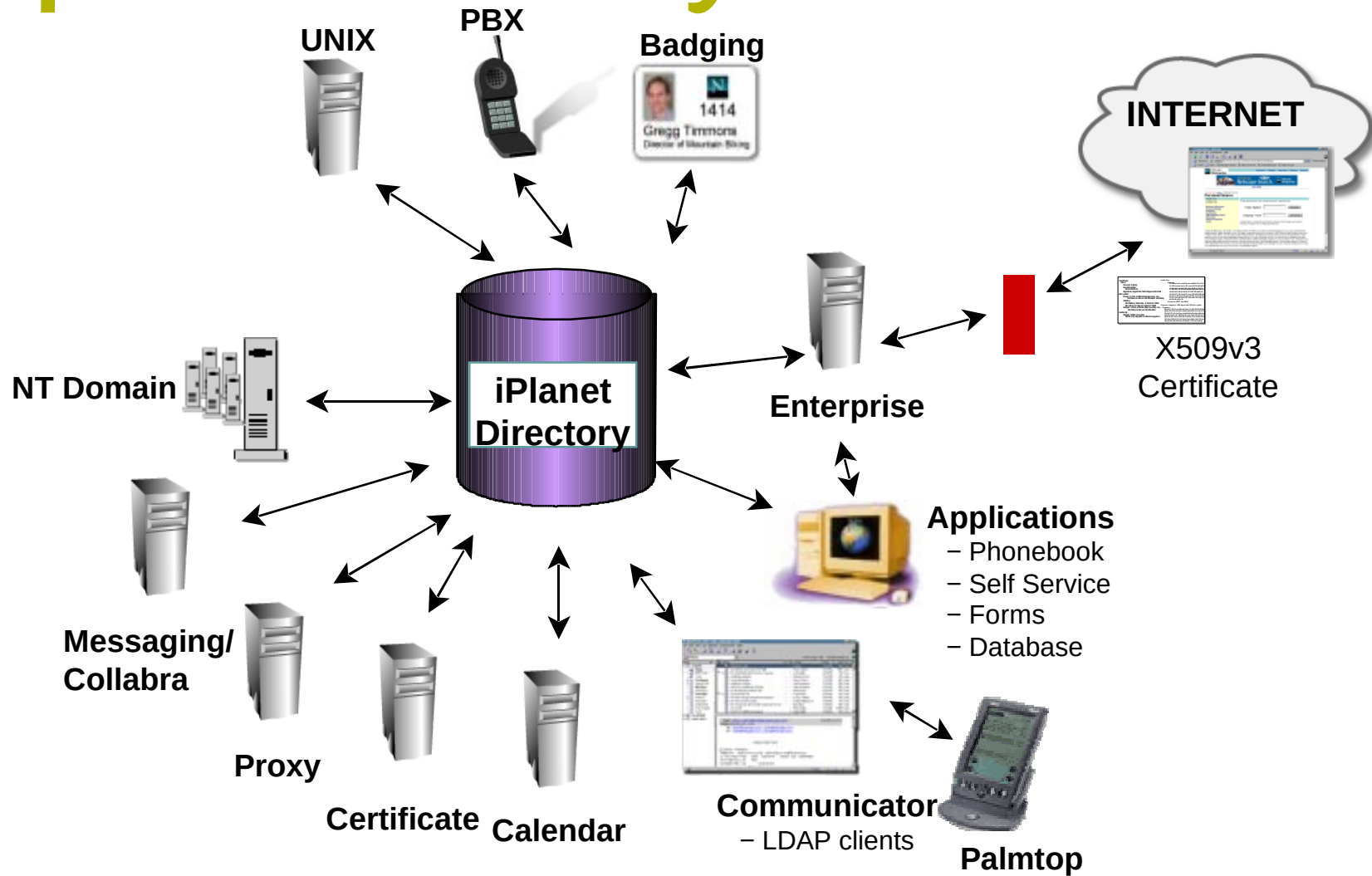
XACML: <http://www.xacml.org>

JCP: <http://jcp.org>

Agenda

- J2SE Security
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

Iplanet Directory Server



Iplanet Security

- Directory and Certificate Products
 - Based around LDAP
 - Iplanet Directory Server
 - Iplanet Directory for Secure E-Commerce
 - iPlanet Directory Server
 - iPlanet Certificate Management System
 - iPlanet Delegated Administrator
 - iPlanet Meta-Directory

Solaris Security

- Solaris IPsec
- Strong User Authentication
- RBAC - Role Base Access Control
- Solaris Security Toolkit (JASS)
- Solaris Fingerprint Database
- Trusted Solaris (C2 -> B1)

Solaris IPsec

- Today
 - IPv4
 - AH – authentication Header (2402)
 - ESP- Encap. Security Payload (2406)
 - Policy management
 - Manual keying
- In Solaris 9
 - IPv6 support
 - IKE (2407–2409)

Solaris IPsec

- 128 bit MD5 and SHA-1
 - For datagram authentication and integrity
- 56-bit DES and 168-bit Triple DES
 - For payload encryption
- Using IPsec
 - Existing apps use ipsecconf file
 - ipsecconf – IPsec policy configuration file
 - New applications use per-socket policy
 - IPsec policy set by setsockopt()

Strong User Authentication

- PAM
 - Pluggable Authentication Module
- Smart Card Authentication
- SEA
 - Solaris Enterprise Authentication
 - Kerberos v5 based
 - Client side integration using PAM

Overview of RBAC in SolarisTM

- Role-Based Access Control
 - Assign limited admin capabilities or other capabilities to users or applications
 - Audit users and roles
 - API to create RBAC aware applications
- GUI to manage RBAC – SMC
 - SMC is an RBAC aware application

Overview of RBAC in SolarisTM

- RBAC includes these features:
 - Authorization - A right that is used to grant access to a restricted function
 - Execution profile (or simply profile) - A bundling mechanism for grouping authorizations and commands with special attributes; for example, user and group IDs
 - Role - A special type of user account intended for performing a set of administrative tasks

Solaris Security Toolkit(JASS)

- Toolkit to Harden a Solaris installation
- Based on Jumpstart
 - Can be used either with Jumpstart
 - Or on a standalone system
- Security policies based on the Solaris Security Blueprint books

Solaris Fingerprint Database

- Verify the integrity of System
 - Stores a cryptographic checksum of Solaris binaries
 - Available through SunSolve

Trusted Solaris

- Based on Solaris 8
 - Runs Solaris 8 application
 - Adds MAC and Labeling of Data
 - Adds Privileges and Authorizations
 - Can be configured to a variety a security levels from C2 to B1

Agenda

- J2SE Security
- J2ME Security
- J2EE security
- XML Security
- Platform Security
- Status, Resources, Conclusions, Q&A

Java and Web Services Security Status

	Version	Features
Java 2	1.3	RSA support
JAAS	1.0	
JCE	1.2	1.2.1 export
JSSE	1.0.2	Global export
J2ME		https support
J2EE	1.2	Interoperability
XML Security	1.0.2	JSRs

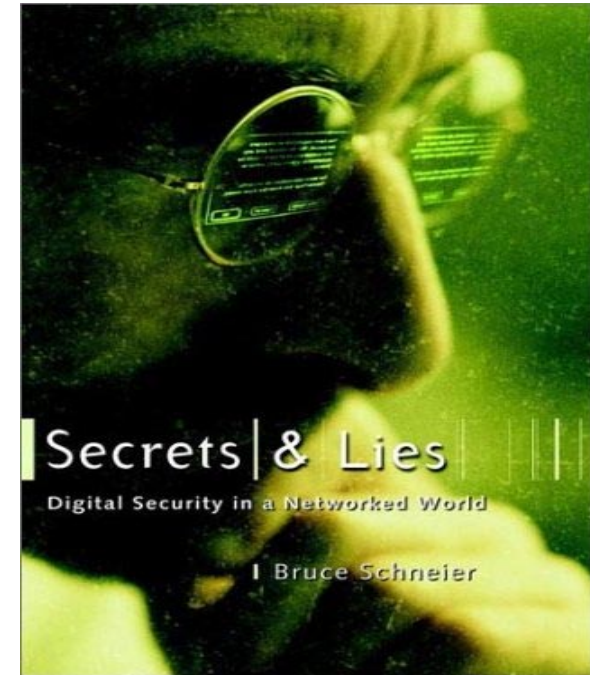
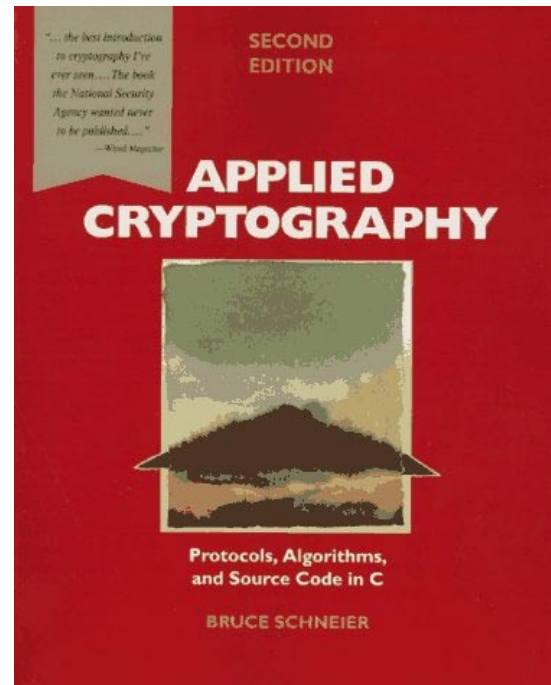
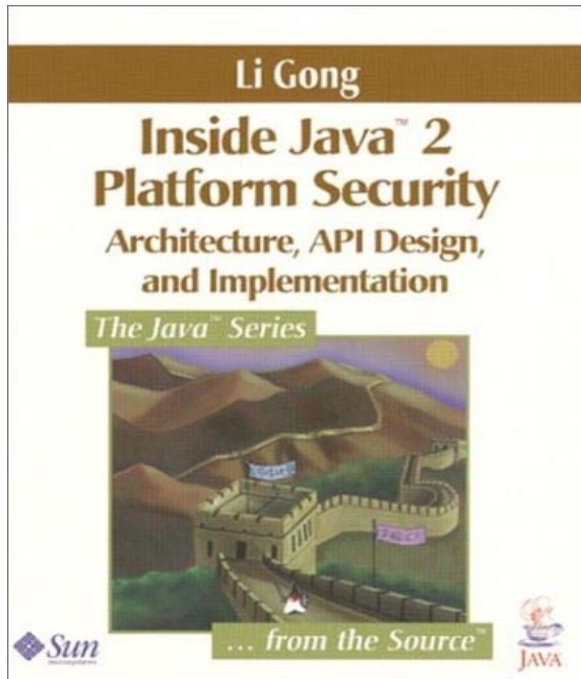
Resources

- Java Security
<http://java.sun.com/security>
- J2EE Security specifications
<http://java.sun.com/j2ee>
- Mastering EJB (including security)
<http://www.theserverside.com>
- J2ME Specifications
<http://java.sun.com/j2me>
- IP
anet Security
<http://developer.ipplanet.com/tech/security>

Resources

-

Books



Articles, Whitepapers, etc.

- JavaWorld Articles
 - Java Security parts 1, 2, 3 and 4 by Raghavan N. Srinivas
 - Advanced Encryption Standards by Raghavan N. Srinivas

<http://www.javaworld.com>

- Audiocasts at

<http://java.sun.com/jdc/onlineTraining/webcasts>

- Articles, whitepapers, tutorials at

<http://java.sun.com/security>

Call for Action!

- Download J2SE 1.4 early Access!
- Optional packages are integrated into core
- J2EE specifications promote security interoperability
- Target J2ME-NG with https
- Look for J2SE/XML Security APIs

www.sun.com/developers/evangcentral

In pursuit of the best software in the universe

- ⑤ All presentations
- ⑤ Audiocasts
- ⑤ Codecamp materials
- ⑤ Technology briefings
- ⑤ code/articles/links/chats/resources



Conclusions

- Security is **built into** the platform
- Delivers portable security with fine-grained access control
- Provides building blocks for secure environments
- Optional packages supplement core security and provide transparent usage



SUN™ TECH DAYS

A DEVELOPER CONFERENCE 2001•2002

Raghavan N. Srinivas

rags@sun.com

